

CAPABILITY DEVELOPMENT PROGRAM

AI Engineering & Model Development Track

Part 2 — Applied LLM, RAG, Agents & Production AI

Program parameter	Detail
Track	AI Engineering & Model Development — Part 2
Format	Instructor-led, hands-on, live lab-based delivery
Duration	4 sessions × 3.5 hours = 14 contact hours
Prerequisite	Part 1 — Python, ML and Deep Learning Engineering Foundations
Audience	Engineers building and operating AI systems in production
Delivery ratio	Approximately 35% concept, 65% hands-on build
Assessment	Lab checkpoints, capstone build, Shark Tank pitch, final assessment
Outcome	Certificate of completion with performance tiering

Every module is delivered against a running enterprise use case. Learners leave with working code, not slides.

1. Where Part 1 Left Off

Part 1 built the engineering base required to write, test and ship AI code. Part 2 assumes fluency in everything below. This section is a checkpoint, not new teaching.

Part 1 module	Covered at a high level
Python & Engineering Foundations for AI Systems	• Async Python with asyncio for high-throughput AI workloads • Pydantic for data validation, schema enforcement and output parsing • FastAPI for production-grade AI inference endpoints • Modular, testable AI application code structure • LLM SDKs: OpenAI, Anthropic, Gemini, HuggingFace • Environment management, secrets handling and config practice • Git-based workflow — branching, versioning, PR standards
Machine Learning Engineering — Applied Production Skills	• Feature engineering and preprocessing pipelines with Scikit-learn • Supervised and unsupervised models — when LLMs are overkill • Experiment tracking with MLflow and Weights & Biases • Hyperparameter tuning — Optuna, grid search, early stopping • Evaluation beyond accuracy — precision, recall, F1, AUC • Class imbalance, data drift and distribution shift • Packaging and serving classical ML models via REST APIs
Deep Learning & Neural Network Engineering	• PyTorch — tensors, autograd, forward and backward pass • Transformer architecture — attention, tokenization, positional encoding • Building and training neural networks from scratch • CNNs for vision; LSTMs versus Transformers for sequences • GPU memory management and mixed-precision training (FP16/BF16) • Overfitting controls — dropout, weight decay, early stopping • Checkpointing, saving, loading and ONNX export

Carry-forward into Part 2

Async, Pydantic and FastAPI are used from Day 1 onward — all APIs built in Part 2 are async and schema-validated. Transformer internals from Part 1 are the basis for fine-tuning, quantisation and serving decisions in Part 2. MLflow / W&B experiment discipline carries into prompt versioning, RAG tuning and evaluation runs.

2. Part 2 at a Glance

Part 2 covers seven capability areas across four sessions. Load is deliberately balanced so each day has one dominant build and one supporting theme — no day is over-packed, and every day ends with working, running code.

Day	Theme	Capability areas covered	What learners walk out with
Day 1	Foundation Models & Model Customisation	• LLM Engineering — working with foundation models • Fine-tuning, alignment and model customisation	A provider-agnostic LLM service with routing, structured output and a first fine-tuned adapter
Day 2	Retrieval & Multimodal Intelligence	• RAG systems — build and optimise end-to-end • Multimodal AI engineering	A measurably optimised hybrid RAG pipeline extended to scanned and image-bearing documents

Day	Theme	Capability areas covered	What learners walk out with
Day 3	Agent Engineering	- AI agent engineering — tools, memory, reasoning loops - Multi-agent orchestration and MCP integration	A stateful, tool-using, memory-backed agent connected to internal systems via MCP
Day 4	Production, Evaluation & Responsible AI	- LLMOps — deploying and operating AI in production - AI evaluation, safety and responsible engineering	A containerised, observable, eval-gated deployment with guardrails and a red-team report

Standard session shape (3.5 hours)

Block	Duration	Purpose
Block A	≈ 25 minutes	Concept framing, architecture patterns, decision criteria and anti-patterns
Block B	≈ 95 minutes	Primary guided build — instructor-led, learners coding along in their own repo
Break	10 minutes	—
Block C	≈ 65 minutes	Second build or optimisation pass on the same artefact, with measurement
Block D	≈ 15 minutes	Debrief, failure-mode review, checkpoint submission and next-day setup

Delivery principles

One running enterprise use case across all four days — each day extends the previous day's artefact rather than starting fresh.

Every hands-on ends with a measurement, not just a working script: cost, latency, retrieval quality, eval score or guardrail coverage.

Instructor demonstrates failure first, then the fix — learners are expected to break their own builds.

Day 1 establishes the engineering layer between an application and any foundation model — provider abstraction, cost and latency control, structured output — and then answers the question every team eventually asks: prompt, retrieve, or fine-tune? Two labs only; every topic in a block is exercised in the lab beside it.

Session block	Topic area	Topics covered	Hands-on
Block A 0:00 – 1:40 25 min concept + 75 min lab	Model access layer	• LLM API integration: OpenAI, Anthropic, Gemini, Amazon Bedrock, Vertex AI • Streaming responses and async call patterns • Retry, timeout and cross-provider fallback	HANDS-ON LAB 1 · 75 MIN Unified LLM gateway One async interface across all five providers, then instrumented and made cost-aware. • Provider-agnostic async client with streaming • Retry, timeout and automatic fallback to a second provider • Token and cost meter; TTFT and total latency profiler • Context-window handler: chunk, summarise, slide • Rule-based router; measure cost cut on a fixed workload
	Cost and latency	• Token budgeting and cost-per-call modelling • Latency profiling — time to first token versus total completion	
	Context and routing	• Context window management: chunking, summarisation, sliding window • Model routing — sending each task to the cheapest capable model	
Break — 10 minutes (1:40 – 1:50)			
Block B 1:50 – 3:20 20 min concept + 70 min lab	Structured output	• JSON mode, function calling and tool use across providers • Pydantic schema enforcement and repair-on-failure	HANDS-ON LAB 2 · 70 MIN Extraction service and first adapter A schema-enforced service with a prompt regression gate, then a first tuned adapter. • Extraction endpoint returning schema-validated JSON • Prompt registry plus a regression suite that fails the build • Synthetic SFT dataset generated from the use case • QLoRA adapter trained on a small open model • Base versus tuned compared on an LLM-as-Judge rubric
	Prompt lifecycle	• Prompt versioning and a Git-backed prompt registry • Regression testing pipelines for prompt changes	
	Model customisation	• Decision framework: prompt versus RAG versus fine-tune • SFT dataset preparation — quality over quantity • Synthetic data generation without labelled datasets • LoRA and QLoRA adapter training; DPO for output alignment • Tooling: HuggingFace TRL, Unsloth, LLaMA-Factory	
	Evaluating tuned models	• MT-Bench, MMLU and LLM-as-Judge scoring	
3:20 – 3:30 · Wrap — failure-mode review, checkpoint submission, Day 2 setup			

Day 1 outcomes

- Can integrate, stream from and fail over between five major model providers using one interface.
- Can model and defend the cost and latency profile of an AI feature before it ships.
- Can guarantee machine-readable output and detect prompt regressions automatically.

- Can decide with evidence whether a problem needs a better prompt, retrieval or a tuned adapter.

Failure modes this day removes

Common failure mode	Where it is addressed
Application locked to one model provider	Lab 1 — one interface with cross-provider fallback
Token spend discovered only on the invoice	Lab 1 — cost meter, budgets and a cheapest-capable router
Model output parsed out of free text	Lab 2 — JSON mode and Pydantic schema enforcement
Prompt edits shipped without testing	Lab 2 — prompt registry and regression suite gating the build
Fine-tuning reached for before it is needed	Block B — decision framework applied before any training run

Tooling and environment — Day 1

Category	Tools and services used
Model providers	OpenAI, Anthropic, Gemini, Amazon Bedrock, Vertex AI
Application layer	Python 3.11, asyncio, httpx, Pydantic, FastAPI
Cost and latency	tiktoken, custom token and cost meter, latency profiler script
Prompt management	Git-versioned prompt registry, pytest-based regression suite
Fine-tuning	HuggingFace TRL, Unsloth, LLaMA-Factory, PEFT; one cloud GPU per team
Evaluation	LLM-as-Judge harness, MT-Bench-style rubric

Pre-session setup — Day 1

API keys issued for all five providers; Part 1 repository scaffold cloned and running.
 One cloud GPU instance provisioned per team for the adapter training run.
 Sample enterprise document set and the running use case brief distributed in advance.

Checkpoint deliverable — Day 1

Lab 1 artefact: async multi-provider LLM gateway with routing and cost metering, committed to the team repository.
 Lab 2 artefact: schema-enforced extraction service plus one trained LoRA/QLoRA adapter with a before-and-after evaluation note.

Day 2 is one pipeline: built in Lab 1, then optimised and extended in Lab 2. Every improvement is proved with a retrieval or evaluation score rather than asserted — and the same pipeline is then pushed to handle scanned, tabular and image-bearing documents.

Session block	Topic area	Topics covered	Hands-on
Block A 0:00 – 1:40 25 min concept + 75 min lab	Ingestion	- Document ingestion pipelines — parsing PDFs, HTML, tables and images - Layout-aware parsing and table extraction - Metadata design for filtering and attribution	HANDS-ON LAB 1 · 75 MIN Ingest, index, retrieve A working pipeline over a messy mixed-format corpus, with a measured baseline. - Ingestion of PDFs, HTML, tables and images with metadata - Three chunking strategies compared on the same documents - Two embedding models benchmarked on a domain query set - Same corpus indexed in pgvector and in Qdrant - Baseline retrieval score recorded as the day's benchmark
	Chunking	Chunking strategies: fixed, semantic, recursive and late chunking	
	Embeddings and stores	- Embedding models: OpenAI, Cohere, BGE, Jina — selection and benchmarking - Vector databases: Pinecone, Qdrant, Weaviate, pgvector — setup and operations	
Break — 10 minutes (1:40 – 1:50)			
Block B 1:50 – 3:20 20 min concept + 70 min lab	Retrieval quality	- Hybrid search: BM25 plus dense vectors - Reciprocal Rank Fusion and score normalisation - Re-ranking with cross-encoders — Cohere Rerank, BGE-Reranker	HANDS-ON LAB 2 · 70 MIN Optimise, evaluate, go multimodal The same pipeline improved in measured passes, then extended past plain text. - BM25 added and fused with dense retrieval using RRF - Cross-encoder re-ranker layered on; lift measured per stage - RAGAS scoring of naive versus optimised pipeline - Scanned invoices and chart images added via VLM extraction - Multimodal index queried across text and image in one path
	RAG evaluation	- RAGAS: faithfulness, answer relevance, context recall and precision - Using evaluation results to drive the next optimisation	
	Multimodal models	- Vision-language models: GPT-5 and 4o, Gemini 2.5, LLaVA, Qwen — integration patterns - Document AI — structured extraction from PDFs, forms and invoices - Image-to-text pipelines for enterprise document processing	
	Multimodal retrieval	- Embedding and retrieving across text and image - Pipeline assembly with LangChain and LlamaIndex - Handling multi-format inputs on a single inference endpoint	
3:20 – 3:30 · Wrap — cross-team score comparison, checkpoint submission, Day 3 setup			

Day 2 outcomes

- Can build an ingestion and chunking pipeline that survives real enterprise document formats.

- Can select embeddings and a vector store on measured evidence rather than defaults.
- Can raise retrieval quality with hybrid search and re-ranking and prove the lift numerically.
- Can extend a text RAG system to scanned, tabular and image-bearing documents.

Failure modes this day removes

Common failure mode	Where it is addressed
Chunk size chosen by default and never revisited	Lab 1 — three chunking strategies compared on one corpus
Embedding model picked off a public leaderboard	Lab 1 — benchmarked on the team's own domain queries
Dense-only retrieval missing exact identifiers	Lab 2 — BM25 fused with dense vectors using RRF
Answer quality judged by reading a few outputs	Lab 2 — RAGAS scoring against a fixed query set
Scanned and tabular documents quietly dropped	Lab 2 — VLM extraction and a multimodal index

Tooling and environment — Day 2

Category	Tools and services used
Parsing and ingestion	PyMuPDF, unstructured, layout-aware table extraction
Orchestration	LangChain, LlamaIndex
Embeddings	OpenAI, Cohere, BGE, Jina
Vector stores	pgvector on Postgres, Qdrant (Pinecone and Weaviate demonstrated)
Ranking	BM25 via rank_bm25 or OpenSearch, Cohere Rerank, BGE-Reranker
Evaluation	RAGAS with a domain-specific query set
Multimodal	GPT-4o/5 vision, Gemini 2.5, Qwen-VL; LLaVA for local inference

Pre-session setup — Day 2

Mixed-format corpus supplied: native PDFs, scanned pages, HTML, tables and chart images.
 Postgres with pgvector and a Qdrant instance running per team; embedding and rerank keys issued.
 A 30-question domain evaluation set prepared for baseline and post-optimisation scoring.

Checkpoint deliverable — Day 2

Lab 1 artefact: ingestion pipeline and dual-store index with a baseline retrieval score.
 Lab 2 artefact: optimised hybrid pipeline with a RAGAS score sheet and a working multimodal ingestion path.

Day 3 builds an agent from its parts rather than from a framework template. Lab 1 produces a working stateful agent; Lab 2 gives it memory, connects it to internal systems through MCP, adds a second agent, and then deliberately breaks it to build the reliability layer.

Session block	Topic area	Topics covered	Hands-on
Block A 0:00 – 1:40 25 min concept + 75 min lab	Agent anatomy	- Tools, memory, reasoning loop and output validation - Agent versus workflow versus single call — cost and reliability trade-offs	HANDS-ON LAB 1 · 75 MIN Tools and a stateful graph agent An agent assembled from first principles, not from a template. - Four domain tools with Pydantic schemas - Model-free test harness validating each tool - LangGraph workflow with conditional routing and a bounded loop - Checkpointing plus a human-in-the-loop interrupt - Process killed mid-run and resumed from checkpoint
	Tool engineering	- Designing, registering and documenting tool functions - Output validation on tool results - Testing tools independently of the model	
	LangGraph	- Graph state, nodes, edges and conditional routing - Cyclical reasoning loops with controlled termination - Checkpointing, resumability and human-in-the-loop interrupts	
Break — 10 minutes (1:40 – 1:50)			
Block B 1:50 – 3:20 20 min concept + 70 min lab	Memory architecture	- In-context versus external memory - Session, episodic and long-term memory design - External stores: Redis, Postgres, pgvector - Memory retrieval, decay and context-budget management	HANDS-ON LAB 2 · 70 MIN Memory, MCP and reliability The agent made production-shaped, then deliberately broken and hardened. - Redis session memory and pgvector long-term memory - Recall verified across process restarts - Internal API exposed as an MCP server and connected - Second specialist agent added with a defined handoff - Whisper voice input channel wired in - Runaway loop and tool failure induced, then contained
	Multi-agent systems	- CrewAI and AutoGen — roles, handoffs and supervision - When multi-agent helps and when it multiplies cost and failure	
	MCP and input channels	- MCP integration — connecting agents to internal APIs and data sources - Speech-to-agent workflows using Whisper transcription	
	Reliability	- Error handling and graceful degradation - Loop detection, step budgets and cost ceilings - Retry logic, idempotency and safe tool re-execution	
3:20 – 3:30 · Wrap — failure-mode review, checkpoint submission, Day 4 setup			

Day 3 outcomes

- Can build an agent from first principles and explain every component in its loop.
- Can design and test tools that models invoke correctly and safely.

- Can make an agent stateful, resumable and memory-backed across sessions.
- Can connect an agent to internal systems through MCP and contain its failure modes.

Failure modes this day removes

Common failure mode	Where it is addressed
Tools written for humans rather than for models	Lab 1 — schema-first tool design with model-free tests
Agent state lost whenever the process dies	Lab 1 — LangGraph checkpointing and mid-run resume
Entire history stuffed back into the prompt	Lab 2 — external memory with explicit context budgets
Multi-agent adopted before it is justified	Block B — cost and failure trade-off assessed first
Runaway loops and repeated tool invocations	Lab 2 — loop detection, step budgets and cost ceilings

Tooling and environment — Day 3

Category	Tools and services used
Agent framework	LangGraph with checkpointing; CrewAI and AutoGen for multi-agent
Tooling layer	Pydantic tool schemas, pytest tool test harness
Memory	Redis for session state, Postgres with pgvector for long-term memory
Integration	MCP server and client SDK, internal REST API mock
Speech input	Whisper transcription service
Reliability	Step budgets, loop detectors, structured retry middleware

Pre-session setup — Day 3

Day 2 retrieval pipeline available to each team as a callable tool.
Redis and Postgres with pgvector running; internal REST API mock and MCP SDK installed.
Audio samples provided for the speech-to-agent input channel.

Checkpoint deliverable — Day 3

Lab 1 artefact: checkpointed LangGraph agent with validated, independently tested tools.
Lab 2 artefact: memory-backed, MCP-connected agent with a documented loop, retry and budget control set.

Day 4 takes the Day 3 agent from working to shippable. Lab 1 serves, containerises, deploys, instruments and caches it; Lab 2 evaluates it, attacks it and guards it. The session closes with the capstone brief.

Session block	Topic area	Topics covered	Hands-on
Block A 0:00 – 1:40 25 min concept + 75 min lab	Serving	- Model serving with vLLM and TGI — throughput, continuous batching, concurrency - GPU utilisation, KV cache and quantisation trade-offs	HANDS-ON LAB 1 · 75 MIN Serve, deploy, observe, cache The Day 3 agent taken from a laptop to a running, instrumented deployment. - Open model served with vLLM and load-tested at rising concurrency - Agent containerised and deployed to Kubernetes - CI pipeline whose evaluation gate blocks a regressing deploy - End-to-end tracing plus latency, token and cost dashboards - Semantic caching added; call-volume and spend savings measured
	Deploy and CI/CD	- Containerising AI applications with Docker - Deploying on Kubernetes — resources, probes, autoscaling - CI/CD with automated evaluation before every deploy	
	Observability and cost	- LLM observability with LangSmith, Phoenix/Arize and Helicone - Monitoring TTFT, token usage, cost drift and output quality - Prompt versioning, A/B testing and shadow deployment - Semantic caching with Redis and GPTCache	
Break — 10 minutes (1:40 – 1:50)			
Block B 1:50 – 3:20 20 min concept + 70 min lab	Evaluation frameworks	- Task-specific test sets and scoring rubrics - LLM-as-Judge — evaluating outputs at scale, and its own failure modes	HANDS-ON LAB 2 · 70 MIN Evaluate, attack, guard The deployment put under scrutiny by its owners and by another team. - Rubric-driven judge pipeline over production-style traffic - Groundedness check added to the retrieval path - Cross-team red-team exercise with logged successful attacks - Guardrails implemented and the attack set re-run - Slice-based fairness test and Responsible AI checklist completed
	Groundedness	Hallucination detection and groundedness checks in production	
	Red-teaming and guardrails	- Adversarial testing, jailbreak probing and edge case discovery - Guardrails: NeMo Guardrails and custom input/output filters - Prompt injection and tool-abuse defence in agentic systems	
	Responsible AI	- Bias detection and fairness testing across demographic slices - Responsible AI checklist before every production deployment	
3:20 – 3:30 · Capstone briefing — themes, teams, deliverables, rubric, Shark Tank format and assessment timeline			

Day 4 outcomes

- Can serve, containerise and deploy an LLM application with sensible throughput and cost characteristics.
- Can gate deployments on automated evaluation instead of manual spot checks.
- Can observe an AI system in production — latency, cost drift, token usage and output quality.
- Can red-team, guardrail and responsibly sign off an AI system before release.

Failure modes this day removes

Common failure mode	Where it is addressed
System runs only on the developer's laptop	Lab 1 — containerised, deployed and load-tested
Deployments approved on a manual spot check	Lab 1 — CI pipeline with an automated evaluation gate
Cost drift noticed weeks after it starts	Lab 1 — cost dashboards and semantic caching
Quality claimed without a test set behind it	Lab 2 — rubric-driven judge pipeline and groundedness checks
Guardrails added only after an incident	Lab 2 — red-team first, guardrail second, re-attack third

Tooling and environment — Day 4

Category	Tools and services used
Serving and load testing	vLLM, Text Generation Inference; Locust or k6 for concurrency runs
Packaging and deploy	Docker, Kubernetes, Helm
CI/CD	GitHub Actions with an evaluation gate on pull requests
Observability and caching	LangSmith, Phoenix/Arize, Helicone; Redis and GPTCache
Safety	NeMo Guardrails, custom input/output filters, jailbreak probe set

Pre-session setup — Day 4

Container registry, Kubernetes namespace and CI runner access provisioned per team.
Observability and caching credentials issued; load-testing tool installed.
Each team's Day 3 agent deployed and reachable for the cross-team red-team exercise.

Checkpoint deliverable — Day 4

Lab 1 artefact: containerised, traced, eval-gated deployment with semantic caching enabled.
Lab 2 artefact: evaluation report, red-team findings with guardrail fixes, and a completed Responsible AI checklist.

3. Capstone Projects

The capstone is not a new project. Teams consolidate the artefacts built across the four days into one deployable enterprise AI system, extended to a chosen theme. Teams of three to four; one week from Day 4 to submission.

Capstone themes

Theme	Problem statement	Mandatory technical elements
Intelligent Document Operations	Ingest mixed-format enterprise documents — scanned, tabular, image-bearing — and answer questions with cited, grounded responses.	• Multimodal ingestion • Hybrid retrieval with re-ranking • RAGAS evaluation • Groundedness guardrail
Autonomous Process Agent	An agent that executes a multi-step internal workflow against real systems with human approval at defined gates.	• LangGraph with checkpointing • MCP-connected internal API • Persistent memory • Loop and cost controls
Domain-Specialised Assistant	A customised model that responds in the organisation's format, tone and policy constraints at low cost.	• SFT or DPO adapter • Synthetic dataset generation • Model routing • LLM-as-Judge evaluation
Cost-Optimised AI Service	Deliver a high-volume AI capability at the lowest defensible cost per request without quality loss.	• Dynamic model routing • Semantic caching • Latency and cost dashboards • A/B or shadow deployment

Deliverables

Deliverable	Requirement
Working system	Deployed and demonstrable; containerised, running end to end, no local-only builds
Source repository	Clean structure, documented setup, tests, prompt versions committed
Evaluation report	Task-specific test set, scores, before-and-after optimisation evidence
Cost and latency note	Cost per request, TTFT and total latency, with the optimisation decisions taken
Responsible AI pack	Red-team findings, guardrails implemented, completed pre-deployment checklist
Pitch deck	Maximum eight slides, business-first framing for the Shark Tank round

Capstone evaluation rubric

Criterion	Weight	What is assessed
Technical depth and correctness	30%	Architecture quality, correct use of frameworks, code standards, absence of shortcuts
Evaluation rigour	20%	Quality of the test set, honesty of measurement, evidence of iteration driven by results
Production readiness	20%	Deployment, observability, error handling, cost and latency control
Safety and responsibility	15%	Guardrails, red-team results, bias testing, checklist completeness

Criterion	Weight	What is assessed
Business impact	15%	Clarity of the problem solved and defensibility of the value claimed

4. Shark Tank Presentation

Teams pitch their capstone to a panel as if seeking funding to take it into production. Engineering quality alone does not win — teams must defend cost, risk and value under questioning.

Format

Element	Detail
Panel	Senior engineering, delivery and business stakeholders
Pitch	8 minutes — problem, solution, live demo, measured results, cost model
Live demo	Mandatory; recorded walkthroughs are not accepted
Q&A	5 minutes of unscripted panel challenge, including a technical deep-dive question
Team participation	Every team member presents at least one section and answers at least one question
Outcome	Panel scores each team; top teams recognised at certification

Judging criteria

Criterion	Weight	What the panel is testing
Problem framing	20%	Is this a real problem worth solving, stated in business terms
Solution and demo	30%	Does it work live, and is the engineering behind it sound
Evidence and metrics	20%	Are the claims backed by measurement rather than assertion
Cost and scalability	15%	Does the unit economics hold at production volume
Defence under questioning	15%	Depth of understanding when the panel challenges design decisions

What the panel penalises

Demos that fail because the system was never deployed outside a notebook.
Quality claims with no evaluation set behind them.
Agentic systems with no cost ceiling, loop control or guardrail.
Presenters who cannot explain a design decision made by a teammate.

5. Final Assessment

Assessment is continuous across the four days and consolidated at the end. Nothing is assessed by attendance alone.

Component	Weight	Description
Daily lab checkpoints	20%	Four submissions, one per day; assessed on whether the artefact runs and meets the day's requirement
Technical assessment	20%	60-minute assessment covering architecture judgement, debugging and scenario-based decisions across all seven capability areas
Capstone build	35%	Scored against the capstone rubric — technical depth, evaluation rigour, production readiness, safety, impact
Shark Tank pitch	15%	Panel score against the judging criteria
Peer red-team review	10%	Quality of adversarial testing performed on another team's system

Technical assessment structure

Section	Coverage
Scenario decisions	Prompt versus RAG versus fine-tune; model selection; agent versus workflow; build versus buy
Debugging	Given failing RAG output, a looping agent and a cost spike — identify the root cause and the fix
Architecture	Design a production AI system for a stated constraint set, with cost and failure analysis
Safety	Identify vulnerabilities in a given prompt, tool definition and deployment configuration

Grading bands

Band	Aggregate score	Interpretation
Distinction	85% and above	Can independently design, build and take an enterprise AI system to production
Merit	70% – 84%	Can build and deploy production AI features with limited senior review
Completion	60% – 69%	Can contribute effectively to production AI workstreams under guidance
Not yet cleared	Below 60%	One re-attempt permitted on the capstone and technical assessment

6. Certification

Participants who successfully complete the programme receive a certificate for the AI Engineering & Model Development Track.